2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00105

NVMePass: A Lightweight, High-performance and Scalable NVMe Virtualization Architecture with I/O Queues Passthrough

Yiquan Chen[1,2,*], Zhen Jin[1,2,*], Yijing Wang[2], Yi Chen[3], Jiexiong Xu[1], Hao Yu[2], Jinlong Chen[2], Wenhai Lin[1], Kanghua Fang[2], Keyao Zhang[1], Chengkun Wei[1], Qiang Liu[2], Yuan Xie[4], Wenzhi Chen[1]
[1]Zhejiang University, [2]Alibaba Group, [3]University of Michigan, [4]HKUST
Email:zy.zhengyi@alibaba-inc.com, jin_zhen@zju.edu.cn, chenwz@zju.edu.cn

***Abstract*—Most data-intensive applications currently run on NVMe storage, and virtualization is essential in cloud computing. Existing NVMe virtualization technologies include software-based and hardware-assisted. Virtio suffers from severe performance degradation, and polling-based solutions consume too many valuable CPU resources. Hardware-assisted solutions provide high performance and no CPU usage but have the challenges of developing dedicated hardware.**

In this paper, we propose NVMePass, a novel software-hardware co-design NVMe passthrough virtualization architecture designed to achieve high performance and no CPU overhead while maintaining high scalability. The key ideas of NVMePass are NVMe I/O queues passthrough for VMs and a mechanism to ensure security. The NVMePass supports DMA and interrupts remapping for VMs without hypervisor involvement, eliminating virtualization overhead and providing near-native performance. Isolation is achieved by I/O queues and logical block address resources exclusively allocated to VMs. We propose NVMe Resource Domain (NRD) and implement it in the NVMe controller to intercept illegal I/O requests. Thus, isolation and security are fully achieved. Results from our experiments show that NVMePass can provide comparable performance to VFIO, with an IOPS of 100.1%-100.5% of VFIO. Furthermore, compared to SPDK-Vhost, NVMePass achieves 40.0% lower latency when running 150 VMs, and NVMePass has an improvement of 68.0% OPS performance in a real-world application when running 100 VMs.

I. Introduction

Solid state drives (SSDs) offer superior performance in terms of throughput and latency when compared to hard disk drives (HDDs) [11], [12], [39]. Non-volatile memory (NVM) technologies boost the inexorable trend toward creating ever-more powerful storage devices. NVM Express (NVMe) [17] is proposed to harness the immense potential of fast NVM storage, leveraging its advantages of high bandwidth and ultra-low latency [54]. The NVMe interface has become an industry standard for SSDs, and NVMe SSDs are widely deployed in data centers such as AWS [7], Alibaba Cloud [6], [15], Microsoft Azure [37], and Google Cloud [19]. NVMe SSDs have gained widespread adoption across various applications, including cloud storage, elastic computing, databases, and big data within cloud data centers, and their significance is only growing.

*The first author and second author contributed equally to this work.

NVMe virtualization is a cornerstone of data centers, enabling Virtual Machines (VMs) to access physical storage devices. This technology decouples the virtual devices of the VMs from the physical devices on the host, allowing for the flexible mapping of multiple virtual devices to a single physical device [46]. Consequently, multiple VMs can share the host's storage devices, thereby improving the overall resource elasticity of storage devices [24].

Currently, mainstream storage virtualization technologies can be divided into software-based and hardware-assisted solutions.

- Software-based solutions virtualize NVMe devices using host software, allowing them to work with general NVMe devices without requiring additional functionality in the hardware. However, existing software-based solutions suffer from serious performance degradation or high CPU overhead. Specifically, virtio [47] only achieves about 50% of native throughput [43]. Polling-based solutions like SPDK vhost-NVMe [55] and MDev-NVMe [43] have been developed to address the performance issues. These solutions improve I/O performance by using CPU resources to process and submit I/O requests [18], [28], [35], which requires extra valuable CPU resources.
- Hardware-assisted solutions replicate PCIe functions at the hardware level to present virtual NVMe devices for different VMs, such as SSDs with SR-IOV [49] or SIOV [51]capabilities, FVM [30], and BM-Store [16]. This method enables VMs to access hardware resources directly, bypassing the host software stack. As a result, these solutions can achieve high performance without consuming additional CPU resources. However, implementing hardware-assisted solutions necessitates dedicated hardware for NVMe virtualization, which brings extra cost and complexity.

Moreover, previous works neglect the importance of scalability when a single server hosts a huge number of VMs. The scalability is becoming more critical as both the computing power of individual servers and the popularity of lightweight VMs rise. For example, modern two-socket servers can already support up to 384 CPU cores/HTs [10]. Concurrently, lightweight VMs such as Firecracker [1], LightVM [36], and RunD [34]

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00105

significantly boost VM density on a single server. Firecracker can support up to 8000 MicroVMs on a single server. Hence, the NVMe virtualization mechanism should be highly scalable to accommodate the continuously growing number of VMs. Unfortunately, increasing VM density exacerbates high CPU overhead in existing software-based polling solutions and increases hardware complexity for hardware-assisted solutions [25], [32], [58].

In this paper, we propose NVMePass, a novel lightweight and software-hardware co-design NVMe passthrough virtualization architecture designed to achieve high performance and no CPU overhead while maintaining high scalability. It has near-native performance, and the requirements for hardware are the minimum. The key ideas of NVMePass are **NVMe I/O queues passthrough for VMs** and a **mechanism to ensure security**. We designed a new I/O queues passthrough framework, enabling VMs to access I/O queues directly and avoiding the challenges of high CPU overhead for software virtualization. To solve the challenges of passthrough security, we propose an NRD mechanism to intercept illegal NVMe I/O requests.

There are two critical and unique designs for the NVMePass: ❶ I/O queues passthrough framework, and ❷ passthrough security mechanism. Firstly, the I/O queue passthrough is accomplished by creating a ***virtual Controller Memory Buffer (CMB)*** in virtualized NVMe devices and a custom page fault handler to establish a direct address mapping between the I/O queues in the VM and the physical device. The CMB is only used in virtualized devices to facilitate memory mapping and does not require actual NVMe hardware support. Secondly, we propose ***NVMe Resource Domain (NRD)*** mechanism and implement it in the NVMe controller firmware to intercept illegal NVMe I/O requests. NVMe hardware resources are allocated independently, and non-overlapping address regions are assigned to VMs, ensuring robust isolation. Thus, isolation and security are fully achieved.

The NVMePass provides near-native performance. Besides I/O queues passthrough, the NVMePass supports Direct Memory Access (DMA) and interrupts remapping for VMs without hypervisor involvement, eliminating virtualization overhead. We divide NVMe devices into *control resources* and *data resources*. The control resources include PCIe configuration space, base address registers (BARs), and admin queue. The data resources are I/O data request-related, including I/O queues, doorbell registers, interrupt resources, and LBAs. The data resources are passthrough for VMs. The control resources are used to manage NVMe devices and are not involved in I/O requests. So, control resources are emulated through software within the hypervisor and have no impact on I/O performance.

The NVMePass consists of four components: *NP frontend driver*, *NP backend driver*, *NP device manager*, and *NP guarder*. The **NP frontend driver** presents standard virtualized NVMe devices in the VMs, eliminating the need to modify the guest's applications. The **NP backend driver** is responsible for managing NVMe physical data resources. It creates I/O queues and associated doorbell registers for VMs. This configuration allows VMs to directly access I/O queues while ensuring isolation between the I/O queues of different VMs. It also enables guests' DMA transactions and interrupts processing without hypervisor involvement. The **NP device manager** in the hypervisor emulates control resources and combines allocated data resources to virtual full-featured NVMe devices for VMs. The **NP guarder** verifies the LBAs and DMA of each NVMe I/O command submitted by the VMs, thereby preventing malicious tenants from accessing the data not owned by them.

Compared to existing hardware-assisted schemes, such as SR-IOV, and SIOV, the NVMePass has the advantage of being lightweight, simple, and flexible. For the NVMePass, the only requirement for the hardware is to support NRD in NVMe SSD to ensure security. It is done in the NVMe controller firmware and only needs a few hundred lines of C code. The overhead of the NRD is negligible. SR-IOV-capable NVMe devices must have an additional hardware design and implementation. More functions will make hardware design more complex, have higher cost and power consumption, and increase the risks of stability and reliability. SIOV has more requirements for CPU and NVMe SSD hardware features, thus increasing the effort and difficulty of deployment. It is common for cloud vendors to customize the NVMe firmware [15] to achieve better software-hardware co-design. So, the NVMePass is simpler and more flexible to implement in the firmware software than redesigning NVMe hardware.

We evaluated the NVMePass prototype and compared it with existing NVMe virtualization mechanisms, including hardware-assisted VFIO [53], and software-based SPDK vhost-NVMe [55], virtio [47]. The results demonstrate that NVMePass can achieve up to 203.2% IOPS compared to virtio and provides near-native NVMe performance with an IOPS range of 100.1% to 100.5% of VFIO. Furthermore, compared to SPDK-Vhost, NVMePass achieves 40.0% lower latency than SPDK-Vhost when running 150 VMs. In a real-world application, NVMePass has an improvement of 68.0% in OPS performance when running 100 VMs.

In summary, our contributions are as follows:

- **Novel NVMe Virtualization Architecture.** We propose NVMePass, a novel lightweight and software-hardware co-design NVMe virtualization architecture that achieves high performance and scalability without consuming valuable CPU resources.
- **I/O Queues Passthrough.** We enable VMs to directly access NVMe I/O queues and I/O requests are not involved in the hypervisor. This approach ensures that NVMePass provides high performance while maintaining no CPU consumption.
- **Unique NRD Mechanism for Security.** We design and implement the unique NRD in the firmware of the NVMe controller and fully protect the hardware resources from malicious attacks.
- **Implementation and Comprehensive Evaluation.** We implement NVMePass and conduct ample comparison evaluations between NVMePass and other virtualization solutions in terms of I/O performance, scalability, over-

head, and fairness. Both synthetic benchmarks and real-world application results demonstrate the superiority of NVMePass over other solutions.

II. Background

This section gives background on NVMe and existing NVMe virtualization solutions.

A. NVM Express

NVMe [17] is proposed to exploit the potential of fast NVM storage, which optimizes I/O paths with a large number of I/O queue depths. The NVMe specification allows one Admin Queue Pair to manage the entire controller, handling functions like creating I/O Queue Pairs and namespace management. Up to *65,535 I/O Queue Pairs* can be associated with each Admin Queue Pair, specifically for executing I/O commands(read/write). Each Queue Pair (QP) consists of a Submission Queue (SQ) for submitting commands and a Completion Queue (CQ) for receiving completions. SQ and CQ are circular buffers in host memory, shared with the device through DMA. The head or tail pointer of each queue is maintained in its Doorbell Registers (DBs). Furthermore, *Controller Memory Buffer* is a general-purpose read/write memory region on the NVMe device controller that can be utilized for various purposes. For example, storing SQs in CMB enables the NVMe driver to write the entire SQ entry directly to the controller's internal memory space, avoiding fetching the entry from host memory.

B. NVMe Virtualization

1) Software-based virtualization. Software-based storage virtualization mechanisms can be divided into traditional virtio and polling solutions.

Virtio [47] is a virtualization standard designed to provide efficient and standardized I/O device interfaces, facilitating efficient data exchange between the virtual machine and the host. Virtio-blk is a specific implementation of the Virtio standard for storage devices.

Polling approaches, such as SPDK vhost-NVMe [55] and Mdev-NVMe [43], spend dedicated CPU cores to enhance the efficiency of virtio backend drivers, thus achieving high performance. SPDK vhost-NVMe offers the best performance for NVMe devices [55], which runs threads on dedicated CPU cores to poll the virtual NVMe I/O queues in shared memory. Similarly, Mdev-NVMe [43] uses a mediated pass-through mechanism to design active polling for shadowed SQs and CQs and hosts CQs in the kernel to achieve high performance.

2) Hardware-assisted virtualization. VMs can directly access NVMe devices with the VFIO (Virtual Function I/O) framework, which leverages the I/O Memory Management Units (IOMMU) support for DMA and interrupt remapping (e.g., Intel VT-d [22], AMD-Vi [9]). However, VFIO requires the physical devices to be exclusively assigned to a single VM and sacrifices the resource-sharing feature of virtualization.

To enable device sharing among multiple VMs, PCIe SR-IOV [49] is proposed to virtualize physical devices at the hardware level. SR-IOV-capable devices feature a Physical Function (PF) and multiple Virtual Functions (VFs). Each VF is a lightweight PCIe function that can be assigned to different VMs through the VFIO framework. FVM [30] and BM-Store [16] are hardware-accelerated NVMe virtualization solutions and are implemented at additional dedicated hardware. LeapIO [33] offloads the entire storage stack to an ARM System on Chip (SoC), which significantly reduces the computational burden on the host CPU and improves overall system efficiency. Scalable IOV (SIOV) [51] is an I/O virtualization proposal focusing on the efficient and scalable sharing of I/O devices.

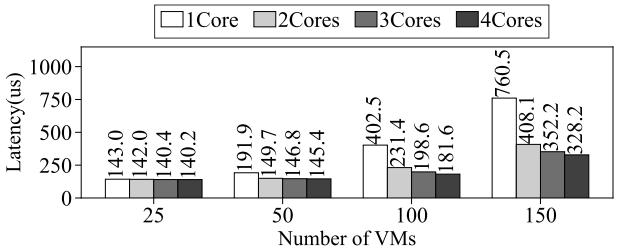


Fig. 1: Average latency of SPDK vhost-NVMe with different CPU cores for polling. As the VM density increases, SPDK vhost-NVMe requires more valuable CPU resources to maintain high performance.

III. Motivation

This section explores the limitations and challenges of existing NVMe virtualization mechanisms, including software-based solutions (virtio and polling-based approaches) and hardware-assisted solutions. The discussion focuses on issues related to I/O performance, CPU overhead, hardware complexity, scalability, and compatibility.

A. Severe Performance Degradation of Virtio

Traditional virtio restricts the high-performance capabilities of NVMe SSDs [43]. For example, Firecracker [5] adopting virtio suffers from more than 50% performance degradation than native NVMe devices. This degradation can be attributed to the costly VM_Exit events, context switching overhead, and cache pollutions [4], [13], [14], [20], [31].

B. High CPU Overhead of Software-based Polling Solutions

Despite providing near-native performance, advanced software-based polling NVMe virtualization (e.g., SPDK vhost-NVMe [55], Mdev-NVMe [43]) incurs high CPU overhead. For example, virtualizing NVMe storage on a server with 16 SSDs consumes 16 CPU cores of servers configured with 128 CPU cores [16], NVMe virtualization overhead can amount to 12.5% of CPU resources. Furthermore, software-based polling solutions have another inherent drawback: as VM density increases, these solutions need more CPU resources to maintain high performance. Fig. 1 illustrates the impact of assigning different numbers of CPU cores for SPDK-Vhost polling. The results demonstrate that when running 150 VMs, the I/O latency of using two cores for polling is 46.3% lower than using only

one core. Therefore, as VM density increases, polling-based solutions require more valuable CPU resources to maintain high performance.

C. Scalability and Complexity of Hardware-assisted Solutions

Many existing hardware-assisted NVMe virtualization solutions are based on PCIe SR-IOV specifications. SR-IOV-capable devices enable hardware virtualization through their acceleration module. However, they have the challenges of hardware complexity and scalability. FVM [30] and BM-Store [16] have to develop additional dedicated hardware to implement virtualization. LeapIO [33] offloads the entire storage stack to the ARM SoC but suffers from severe performance degradation, achieving only 68% [30] throughput due to the limited computing capabilities of the ARM CPU.

Only a few SR-IOV-capable NVMe SSDs can be chosen in the market because SR-IOV is optional in the specification. If cloud vendors use SR-IOV as an NVMe virtualization solution, they will have vendor lock-in and compatibility challenges. Besides, the SR-IOV feature will also increase the design complexity and cost of the NVMe controller and restrict the number of VFs that can be implemented [25], [32], [58].

The SIOV [51] is a hardware-assisted I/O virtualization proposal designed for the hyperscale era and high-density VM workload. However, SIOV requires CPU support for PCIe Process Address Space ID (PASID), and NVMe devices implement an Assignable Device Interface (ADI). ADI is not a part of NVMe specification. Due to these requirements, SIOV has restrictions on specific CPU and NVMe devices.

D. Summary

As we summarize in Table I, for software-based virtualization solutions, traditional virtio solutions have serious performance issues, and polling solutions lead to high CPU overhead. Although hardware-assisted solutions can relieve CPU overhead, they encounter hardware complexity and dedicated hardware challenges.

These challenges motivate us to propose NVMePass, a lightweight and scalable NVMe virtualization architecture that delivers high performance and no CPU overhead. Additionally, NVMePass can be easily implemented and deployed in existing data centers.

IV. Design and Implementation

In this section, we introduce the design of NVMePass and its implementation. We have established the following design goals for NVMePass:

- **High Performance.** Provide near-native performance in throughput and latency, comparable to native NVMe SSDs.
- **Lightweight and No CPU Overhead.** It should not consume valuable CPU resources on the server.
- **High Scalability.** As VM density continues to increase, scalability is vital, as it is common to deploy multiple VMs on a single server with limited NVMe SSDs.
- **Isolation and Security.** VMs can directly submit NVMe commands to I/O queues. Data resources must be isolated among VMs. To ensure security, malicious or illegal I/O requests can be intercepted.

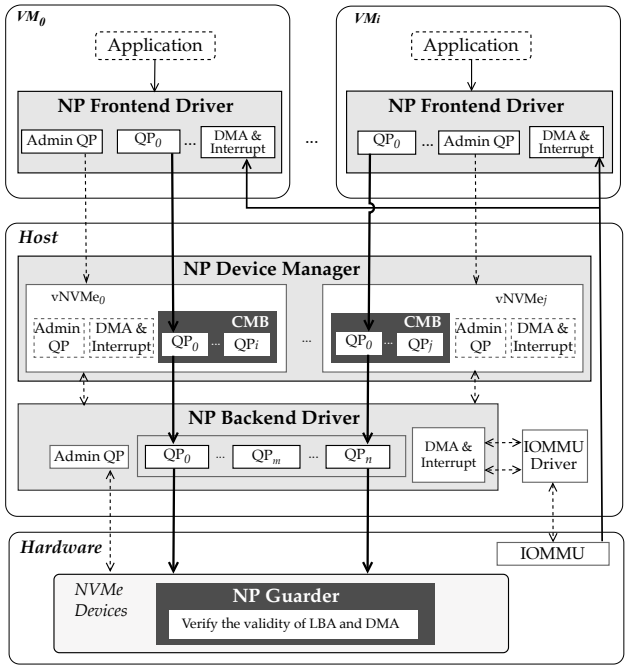


Fig. 2: Overall Architecture of NVMePass

A. Overall Architecture

To meet the aforementioned design goals, we propose NVMePass, a novel NVMe virtualization architecture that enables I/O queues passthrough. Control resources are used in the slow path, so we emulate the control resources of virtual NVMe devices within the hypervisor. Data resources are in the fast path, and we enable VMs to access I/O queues directly. Since the fast path is critical in read-write I/O requests, the NVMePass achieves no CPU overhead and high performance.

Isolation is implemented by data resources exclusively allocated to VMs. Each data resource, such as I/O queues, LBAs, Physical Region Pages (PRPs), etc., belonged to only one VM and had no overlap.

Security is achieved by the firmware of the NVMe controller. VMs directly submit NVMe I/O commands to physical I/O queues, so malicious tenants may create illegal I/O requests to usurp the data that does not belong to them. We define and implement the NP guarder to intercept illegal I/O requests. Thus, the security of the NVMePass is ensured.

Fig. 2 shows the NVMePass architecture and its components. NVMePass consists of four components: NP frontend driver in guest kernel, NP backend driver in the host kernel, NP device manager in the hypervisor, NP guarder in the NVMe devices.

NP frontend driver provides standard NVMe devices for VMs, so user applications or other operating system components need not do any modifications. The NP frontend driver directly leverages the I/O queues in the CMB provided

TABLE I: Comparison of the existing NVMe virtualization and NVMePass

	Software-based			Hardware-assisted					HW-SW
	Virtio [47]	SPDK-Vhost* [55]	Mdev-NVMe* [43]	SR-IOV SSD [38], [48]	LeapIO [33]	FVM† [30]	BM-Store† [16]	SIOV [51]	**NVMePass**
High performance		✓	✓	✓		✓	✓	✓	✔
No CPU overhead				✓	✓	✓	✓	✓	✔
High scalability	✓	✓	✓					✓	✔
Compatibility ‡	✓	✓	✓						✔
Lightweight								✓	✔

* Polling solutions.
† Offloading SR-IOV to dedicated hardware.
‡ Not requiring dedicated hardware.

by the NP device manager to create NVMe I/O queues. Besides, the NP frontend driver is also responsible for converting the virtual LBA to the physical LBA of the NVMe device according to allocated LBA ranges. It also verifies the legitimacy of the destination addresses (LBA and PRP) in I/O requests.

NP backend driver is responsible for allocating I/O queues and enabling VMs to directly access these queues, performing DMA remapping, and handling posted interrupts. It creates I/O queues and maps the I/O queue buffers to VMs via the NP device manager's CMB, facilitating direct access to I/O queues by VMs. Additionally, the NP backend driver allocates LBA for NP device manager. **NP device manager** in hypervisor present standard NVMe devices for VMs by emulating NVMe control resources in software and combining the I/O queues allocated by the NP backend driver. **NP guarder** in the firmware of the NVMe controller verifies the address ranges, including disk and DMA addresses, of the I/O requests issued by the VMs, thereby preventing malicious tenants from accessing or stealing data from others (detailed in subsection IV-C).

B. I/O Queues Passthrough Mechanism

Each VM has its own dedicated I/O queues and related doorbell registers, enabling direct interaction with NVMe devices for issuing I/O requests. This I/O queue passthrough mechanism bypasses the hypervisor and enables zero-copy for I/O requests.

As shown in Fig. 3, the I/O queues passthrough is accomplished through the CMB of the NP device manager and a page fault handler. The NP backend driver utilizes the CMB of the NP device manager to map the I/O queue memory between the host and guest kernel. Notably, the CMB refers to host memory allocated by the hypervisor and has no requirement for implementation of the NVMe devices. The registered page fault handler establishes the `GPA-HPA` mapping for the guest I/O queues in the Extended Page Table (EPT). The details of the NVMe I/O queue passthrough are as follows.

❶ The NP frontend driver in the guest kernel submits I/O requests to the I/O SQ in the CMB, and the Memory Management Unit (MMU) converts the Guest Virtual Address (`GVA`) of the I/O queue to Guest Physical Address (`GPA`). Then, the EPT takes over the `GPA` from the MMU and looks up the

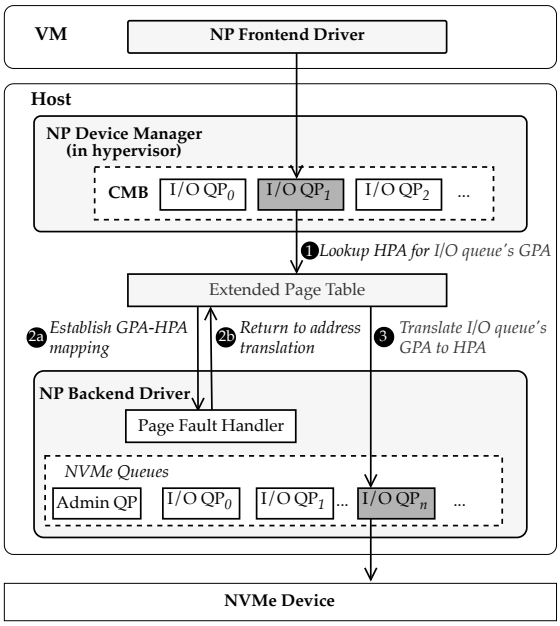


Fig. 3: I/O Queues Passthrough Mechanism

Host Physical Address (`HPA`) for the I/O queue's `GPA` in the EPT. If the `GPA` is not mapped, it will generate a page fault exception, then go to ❷; otherwise, go to ❸. Once page table entries for I/O queues and doorbell registers are established in the EPT, VMs can access NVMe I/O queues directly, like the host. Then, there are no page faults in subsequent I/O requests.

❷ NVMePass implements its own page fault handler. The page fault handler obtains the `HPA` of the corresponding I/O SQ and establishes the `GPA-HPA` mapping in EPT (❷a). After completing the address mapping, it returns to `HPA-GPA` translation in EPT (❷b). The `GPA-HPA` mappings of CQ and doorbell registers are established similarly.

❸ The EPT translates the guest I/O queue's `GPA` to `HPA` according to the established page table entry. So, the VM can access I/O SQ on the host memory to submit the I/O command, which is then fetched and executed by the NVMe device.

The `GPA-HPA` mapping establishment is only executed for

the first request of the page, and no page fault in the future. So, the performance degradation caused by it is negligible. Moreover, the GPA-HPA mapping of I/O queues is established through the page fault handler on the host, and address translation is performed through EPT. This guarantees that VMs are restricted to accessing their own I/O queues exclusively.

C. Isolation and Security

This subsection discusses the isolation and security implementation when a VM directly accesses I/O queues. Isolation ensures that different tenants use independent hardware resources and access distinct address regions. Security mechanisms prevent malicious tenants from usurping other VMs' hardware resources. Potential risks of the *hardware resources* accessed by the VM include NVMe I/O queues, DMA addresses of host memory and LBAs, as well as related doorbell registers.

1) Isolation: NVMePass provides independent hardware resources and non-overlapping address regions to VMs, ensuring robust isolation.

Firstly, LBAs are isolated by the NVMePass device manager and the NP frontend driver. The NP backend driver allocates non-overlapping LBA ranges for different virtualized disks and stores the information in the device's CMB. When I/O requests in the VM are submitted via `nvme_submit_cmd`, the NP frontend driver adjusts the `slba` of the I/O command based on the LBA range information, ensuring that the disk space accessible to different VMs does not overlap.

Secondly, I/O queues and associated registers are isolated by the Extended Page Table (EPT) created by the hypervisor. Each VM has its independent I/O queues and EPT page tables. There are no VMs who have the right to access them.

Lastly, DMA addresses for virtualized disks are also isolated by EPT in the hypervisor. The DMA addresses issued by different VMs for the NVMe device to host memory are mapped to different `HPA`. This ensures that different VMs cannot access the same memory address, as described in IV-D.

2) Security: NVMePass enhances security by isolating I/O queues and doorbell registers of VMs into separate physical pages in the hypervisor, thereby preventing unauthorized access to hardware resources by malicious VMs. Additionally, we implement NP guarder module in the NVMe controller firmware to verify the LBAs and PRPs in each NVMe I/O command to intercept illegal requests from malicious VMs.

I/O Queues and Doorbell Registers Protection. An NVMe submission command is 64 bytes, allowing a 4K page to accommodate 64 NVMe submission commands. A SQ typically has 1024 entries distributed across 16 pages. Similarly, an NVMe completion command is 16 bytes, and 1024 entries CQ is distributed across 4 pages. Therefore, the I/O queues of different VMs are located on separate pages and protected by EPT address translation, preventing malicious VMs from accessing the queues of other VMs.

Besides, Each I/O queue pair (including SQ and CQ) has a SQ tail doorbell register and a CQ head doorbell register, each 4 bytes in size, resulting in a total register size of 8 bytes per I/O queue pair. Given that a 4K page can accommodate 512 I/O queue pair registers, there is a risk that a malicious VM could access the registers of other VMs within the same 4K page through EPT. For this issue, we can increase the spacing between registers inside the NVMe device by setting the `CAP.DSTRD` according to NVMe specification. This ensures that the registers of each I/O queue pair are distributed across independent pages so a VM can only access its register page, thereby enhancing security.

Thus, I/O queues and doorbell registers are fully isolated and protected by EPTs in the hypervisor.

Algorithm 1 Algorithm of NVMePass **NP Guarder** in NVMe Controller Firmware.

```
for each I/O command in Preprocessed List by ASIC of Controller do
    GET LBA/PRP Info from preprocessed NVMe command
    if COMPARE LBA to Pre-allocated Failed then
        return IO_ERROR_LBA;
    end if
    if COMPARE PRP to Pre-allocated Failed then
        return IO_ERROR_PRP;
    end if
end for
return verified NVMe I/O command.
```

LBA and DMA Address Protection. There are chances to usurp the LBAs and PRPs in the NVMe commands, which are directly submitted to physical NVMe I/O queues. Malicious VMs may falsify the NVMe commands by using a modified NVMe driver, replacing NP frontend driver. To this end, we define and implement the NP guarder with the NRD mechanism in the NVMe controller firmware. With minor adjustments to the command validation [8] in the SSD controller, the mechanism can entirely prevent malicious VMs from attempting to access illegal LBAs and PRPs.

The key idea of NP guarder is that we bind LBAs and PRPs with I/O queues to form an NVMe Resources Domain(NRD). Each VM can only access the resources belonging to this NRD. As shown in Fig. 4, NRDs are independent and isolated from each other, and each VM can only access the resources inside its own NRD. On each I/O request issued by VMs, the SSD controller fetches the commands from host memory and verifies whether the target LBA and PRPs belong to the pre-allocated NRD. If not, it returns an I/O error to the VM, and the I/O commands will not be executed. Otherwise, the command is processed normally. As a result, we can fully prevent malicious I/Os and ensure security.

The SSD controller has a command validation module to perform a legitimacy check on the request [23], [26], [29], [44], [50], [57]. Therefore, we only need to add a simple NRD validity compare statement to the module, which is a minor modification. The NP guarder pseudo-code is shown in the algorithm1. The validity check compare statement only costs a few cycles and does not impact I/O performance. Similarly, the NRD validity check can be done in `__nvmev_proc_io` for NVMeVirt [26] and `Cmd Validation` for Xilinx NVMe

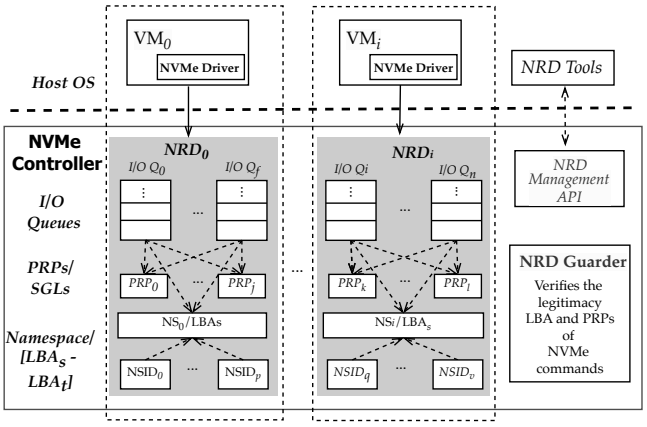


Fig. 4: NP Guarder with NVMe Resource Domain (NRD)

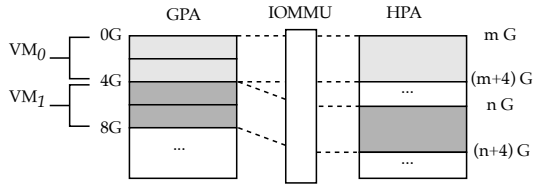


Fig. 5: DMA isolation by assigning non-overlapping GPA addresses to VMs

Target Controller [8]. We implement the NP guarder in the NVMe controller firmware based on Yingren Dongting-N2 NVMe SSD [56]. NRD configurations can be set by NRD tools in the host through the `ioctl` command interface. NP guarder can also be implemented in other NVMe controllers if the firmware's source code is open.

D. DMA Remapping & Posted Interrupt

In addition to submitting I/O requests, efficient DMA data transfer and interrupt handling are essential for a high-performance NVMe virtualization mechanism. To this end, NVMePass handles interrupts without going through costly VM_Exit.

DMA Remapping. The NVMe I/O request contains the PRP or Scatter Gather List (SGL), the physical memory location in the host memory for data transfer. The PRP/SGL address in the guest I/O request is a `GPA`. The `GPA` must be converted to a `HPA` for NVMe devices initiating DMA transactions, and the IOMMU performs `GPA-HPA` address translation automatically.

To guarantee that the PRP/SGL addresses used by virtual NVMe devices in VMs are isolated, the hypervisor sets non-overlapping `GPA` addresses to each VM on initialization. For example, as shown in Fig. 5, the hypervisor allocates `GPA` from 0G to 4G for VM0 and 4G to 8G for VM1. This ensures that the PRP/SGL addresses in the I/O requests issued by VM0 and VM1 are isolated.

Posted Interrupt. To provide efficient interrupt handling, NVMePass combines the IOMMU interrupt remapping unit with the posted-interrupt mechanism. The posted-interrupt mechanism is a hardware mechanism that allows interrupts to be received directly by a VM. In particular, the NP backend driver configures the posted-interrupt I/O queues using the *IRQ bypass manager* [52].

E. NP Device Manager

NVMePass presents virtualized NVMe devices for VMs through the *NP device manager* in the hypervisor. The NP device manager emulates the control resources using software and combines them with data resources allocated by the NP backend driver. NVMePass uses a conventional trap-and-emulate approach to emulate these control resources in the hypervisor.

When the VM accesses the control resources of the NP device manager, it causes VM_Exit events and traps to the host, and then the hypervisor takes over the VM request. Then, the hypervisor reads/updates the relevant virtual registers. Lastly, the hypervisor generates an interrupt to the VM to inform the request completion. In general, control resources are accessed by admin commands for device initialization, management, and status queries. Therefore, using the trap-and-emulate approach for control resources does not affect I/O performance.

The number of virtual NVMe devices that NVMePass can provide for VMs is limited by the number of NVMe I/O queues implemented in SSD. Typically, servers are configured with multiple NVMe SSDs, and one NVMe SSD has multiple queues. According to NVMe specifications [17], the maximum number of I/O queues of one NVMe device is 65,535. As a result, NVMePass can nicely scale as the number of VMs increases.

F. NP Guarder

The NP guarder enforces the security of NVMePass. It implements the NVMe resources domain and verifies the legitimacy of NVMe commands from VMs to prevent malicious or illegal I/O requests. Detailed is discussed in subsection IV-C.

G. Implementation

We implemented NVMePass on the Linux kernel (version 4.19) and a popular hypervisor, Firecracker (version 0.14.0) [5]. NVMePass implementations contain 6,600 LOC in total, including 300 LOC in the guest kernel, 4,200 LOC in the host kernel, and 2,100 LOC in Firecracker. For the guest kernel, we modified the original NVMe driver to the NP frontend driver, which directly leverages the I/O queues in the CMB. In the host kernel, we implemented the NP backend driver based on the original NVMe driver and the VFIO-mdev framework [40]. The NP backend driver creates NVMe I/O queues on the host. Then, it exposes these I/O queues, the doorbell registers, and the LBA range to the hypervisor through the VFIO-mdev framework. In Firecracker, we added support for VFIO devices and adopted the VFIO-mdev framework [40] to provide NVMe data resources for the NP device manager. In addition, we have implemented control resource emulations, including PCIe configuration space, BARs, and NVMe admin queue in Firecracker.

TABLE II: Test Cases

Case name	(bs,rw,numjobs,iodepth)
randread-4k-4-128	(4k,randread, 4,128)
randwrite-4k-4-128	(4k,ranwrite, 4,128)
seqread-128k-4-128	(128k,read,4,128)
seqwrite-128k-4-128	(128k,write,4,128)
randread-4k-1-1	(4k,randread,1,1)
randwrite-4k-1-1	(4k,randwrite,1,1)
seqread-4k-1-1	(4k,read,1,1)
seqwrite-4k-1-1	(4k,write,1,1)

NVMePass can be implemented across multiple NVMe devices. The only requirement for NVMe devices is the implementation of the NP guarder, and there are no other special dependencies, including CMB. Initially, we designed the NP frontend driver, NP backend driver, and the NP device manager for Intel P4510. However, we couldn't access the source code of the P4510 firmware, so we chose Yingren Dongting-N2 [56] NVMe controller to implement the NP guarder. Dongting-N2 is PCIe 4.0 and QLC NAND-based. NP guarder implementations contain 300 LOC.

V. Evaluation

We evaluate NVMePass and compare its I/O performance and scalability with other NVMe virtualization mechanism solutions, including VFIO, SPDK-Vhost, and virtio. We also evaluate the fairness of NVMePass on multiple VMs and run various synthetic benchmarks on real-world workloads.

A. Experimental Setup

System Settings. We deployed NVMePass on a host machine with two 48-core Intel Xeon Platinum 8163 CPUs. To avoid unpredictable fluctuations in test results from accessing devices across Non-Uniform Memory Access (NUMA) [3], we used only one of the NUMA nodes. We used Yingren Dongting-N2, which has a SoC NVMe controller [56] as the NVMe device. For the NVMePass solution, we implemented the NP guarder in the SSD controller to check the legitimacy of the address of the I/O request. The NP guarder module is disabled in none-NVMePass solutions.

Both host and guest operating systems run on 64-bit CentOS with Linux kernel (version 4.19) for implementation. The NP backend driver and NP device manager are installed in the host, and the NP frontend driver is installed in the guest. In addition, we used a modified Firecracker (version 0.14.0) as the hypervisor to manage VMs. We assign 4 I/O queues to each VM for the single-VM scenario, and for multi-VM scenarios, we assign 1 I/O queue to each VM.

Compared Virtualization Solutions. We compared NVMePass with representative NVMe virtualization solutions. VFIO's performance is close to that of the native disk and the best among hardware-assisted solutions. For software-based solutions, we compare NVMePass with SPDK-Vhost and virtio. SPDK-Vhost refers to SPDK vhost-NVMe, which performs best on all SPDK vhost-target solutions.

Benchmarks and Real-world Application. We used the fio [2] as our synthetic evaluation benchmark tool, which allows us to stress-test the disk by specifying specific I/O patterns (including random read and write, sequential read and write, etc.) and different I/O pressure (different threads and queue depth). Specifically, we took libaio as the fio engine and ran various test cases shown in Table II. We bypassed the guest OS kernel page cache by setting the direct parameter to 1. In addition, RocksDB [45], a persistent key-value store for flash and RAM storage, was used to measure the performance of various virtualization solutions in real-world applications.

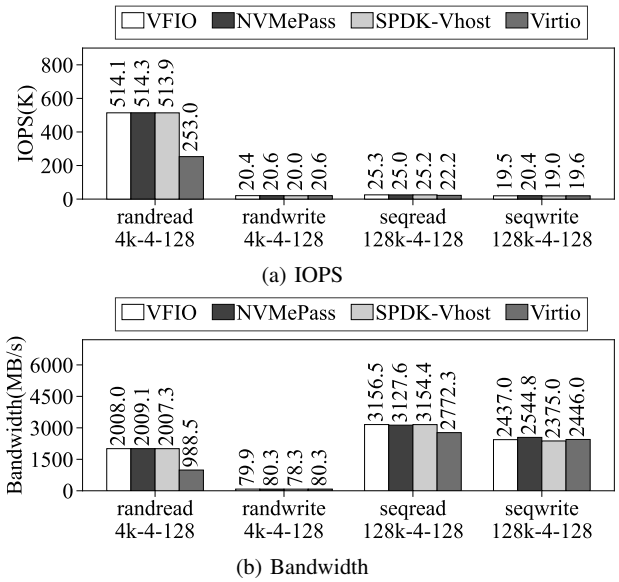


Fig. 6: Single-VM throughput with one SSD

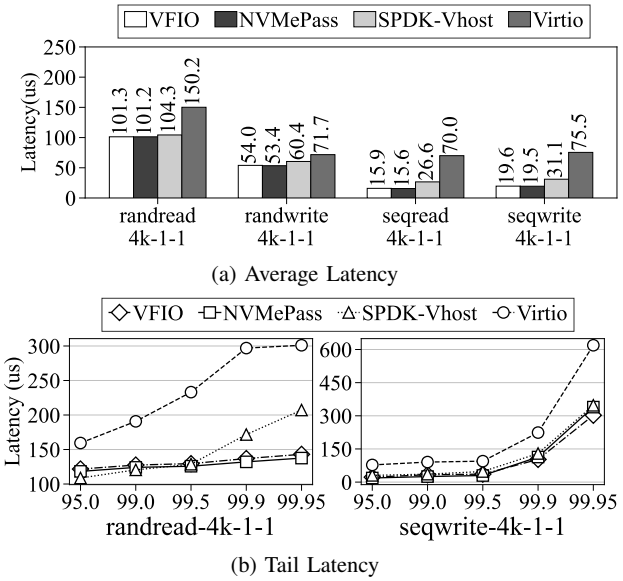
B. I/O Performance

We evaluated the I/O performance of VFIO, NVMePass, SPDK-Vhost, and virtio with one SSD. We ran various test cases shown in Table II on a single VM, which is allocated with 4 CPU cores and 4GB of system memory. For SPDK-Vhost, we allocated one more CPU core for the SPDK vhost-target.

Throughput. Fig. 6 demonstrate the throughput performance of a single VM. NVMePass can provide near-native performance in terms of IOPS and bandwidth.

NVMePass can achieve 100.1% and 100.5% IOPS of VFIO in randread-4k-4-128 and randwrite-4k-4-128, respectively. VFIO passthrough the entire physical device to a VM, which can achieve near-native performance but sacrifice shareability. Compared to SPDK-Vhost, NVMePass has 2.6% higher in IOPS randwrite-4k-4-128. SPDK-Vhost achieves performance close to NVMePass at the cost of consuming valuable CPU cores for polling. Virtio suffers from severe performance degradation and shows the worst virtualized I/O performance. In randread-4k-4-128, NVMePass can achieve 203.2% IOPS of virtio.

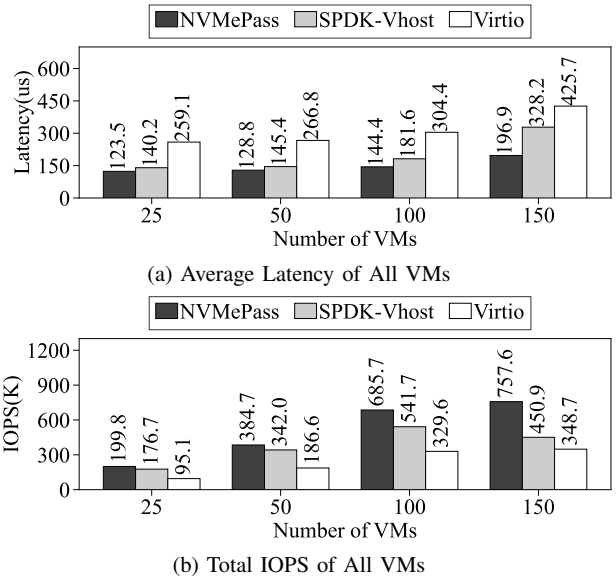
In the 128k sequential read/write test cases, all solutions reach the bandwidth limit of the storage device. NVMePass averagely achieves 99.1% of VFIO, 99.2% of SPDK, and



(a) Average Latency

(b) Tail Latency

Fig. 7: Single-VM Latency with one SSD



(a) Average Latency of All VMs

(b) Total IOPS of All VMs

Fig. 8: I/O performance of multiple VMs running 4k-randread-1-1 on two SSDs

112.8% of virtio. When processing large block requests, there is no significant performance difference among them.

Latency. Fig. 7a depicts the average latency of single-thread and single-qd I/O workloads, and the tail latency of randread-4k-1-1 and seqwrite-4k-1-1 workloads are shown in Fig. 7b.

As seen from Fig. 7a, The average latency difference between NVMePass and VFIO is only less than 1% and can be negligible. It indicates that NVMePass can achieve the same low latency as VFIO, and the NVMePass guarder inside the SSD controller does not affect the I/O performance. The latency of SPDK-Vhost is up to 70.2% higher than NVMePass in seqread-4k-1-1 because SPDK-Vhost has a longer I/O path than NVMePass. In NVMePass, VMs send the I/O requests directly to the device without additional software processing. As a result, NVMePass achieves lower I/O latency than SPDK-Vhost. Virtio exhibits latency drawbacks due to its severe virtualization overhead, suffering from 34.2%-347.6% higher latency than NVMePass. In addition, we can see from Fig. 7b that NVMePass exhibits the best tail latency, followed by VFIO and SPDK-Vhost, and finally, virtio.

To summarize, NVMePass can provide *high performance* in terms of both throughput and latency. Its performance is comparable to VFIO, slightly better than SPDK-Vhost, and far better than virtio. It is worth noting that SPDK-Vhost consumes one more CPU core for polling.

C. Scalability

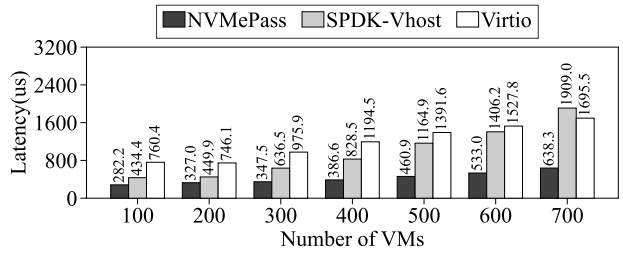
For the scalability test, we ran from 25 to 150 VMs to evaluate NVMePass, SPDK-Vhost, and virtio. Four CPU cores are allocated for SPDK-Vhost polling. We allocated one core for every five VMs. We configured the server with four SSDs, and each VM owns 10G capacity. We ran 4k-randread-1-1 on these VMs and measured the average latency and overall throughput of all VMs.

Fig. 8a presents the latency of NVMePass, SPDK-Vhost, and virtio with different VM densities. Notably, NVMePass stands out with the lowest latency across all VM densities. As the VM density increases, the performance advantage of NVMePass over SPDK-Vhost becomes more significant. In contrast, SPDK-Vhost consumes four more CPU cores than NVMePass. As the number of VMs escalates from 25 to 150, NVMePass provides from 12.0% to 40.0% lower latency than SPDK-Vhost, which can be attributed to the I/O queues passthrough feature of NVMePass. In addition to outperforming SPDK-Vhost, NVMePass also displays a substantial latency advantage over virtio. Specifically, when running 25 VMs, NVMePass's latency is 52.4% lower than that of virtio.

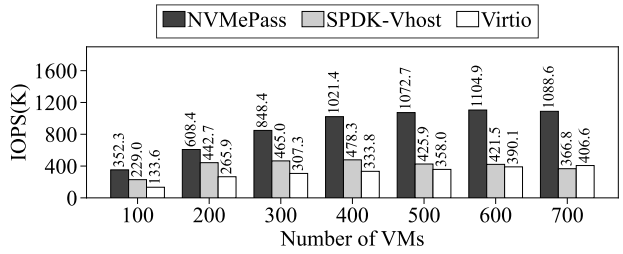
The results presented in Fig. 8b demonstrate that NVMePass outperforms both SPDK-Vhost and virtio in terms of overall IOPS performance in high-density VMs. Specifically, when running 150 VMs, NVMePass provides 68.0% and 117.2% higher IOPS than SPDK-Vhost and virtio, respectively.

Since the Yingren Dongting-N2 SSD is limited to 64 I/O queues, we evaluated the scalability of various virtualization solutions in a highly virtualized environment using the Intel P4510 SSD, which supports 128 I/O queues. As shown in Fig. 9, the number of virtual machines was scaled from 100 to 700, utilizing a total of six SSDs. For every 100 virtual machines, 10 CPU cores were shared, while six CPU cores were exclusively allocated to SPDK-Vhost. The experimental results demonstrate that NVMePass achieves superior scalability compared to virtio and SPDK-Vhost, maintaining optimal latency and IOPS in high-density VM scenarios.

To summarize, NVMePass has superior I/O performance

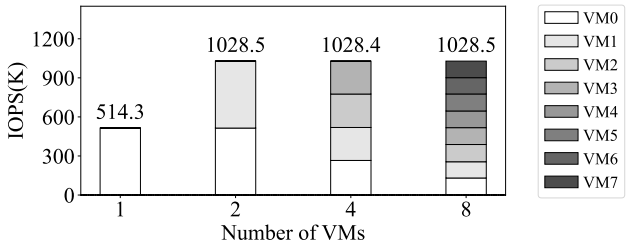


(a) Average Latency of All VMs

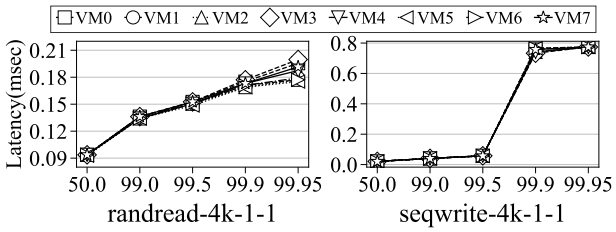


(b) Total IOPS of All VMs

Fig. 9: I/O performance of multiple VMs running 4k-randread-1-1 on six Intel P4510 SSDs



(a) Overall IOPS of multiple VMs running on 2 SSDs



(b) Tail latency distribution of eight VMs

Fig. 10: Fairness of NVMePass

compared to SPDK-Vhost and virtio as the number of VMs increases, exhibiting *high scalability*.

D. CPU Consumption

In this section, we utilized the `rate_iops` parameter of fio to control the I/O pressure on a single VM. The test case was 4k-randread-4-128, with the I/O pressure ranging from 10K to 400K IOPS. We measured the CPU resources consumed by the hypervisor and host processes assisting the VM in I/O operations (such as the SPDK vhost-target).

TABLE III: Consumed CPU cores of NVMe virtualizations under different I/O pressure

I/O Pressure (K IOPS)	10	50	100	200	400
VFIO	0	0	0	0	0
NVMePass	0	0	0	0	0
SPDK-Vhost	1	1	1	1	1
Virtio	0.17	0.63	1.07	1.94	x

Table III demonstrates the CPU consumption of various storage virtualization schemes.

The results reveal that both the VFIO and NVMePass do not consume additional CPU cores on the host during virtualization. The two solutions' I/O commands bypass the hypervisor entirely and are directly transmitted from the VM to the physical NVMe device. In contrast, the SPDK-vhost solution necessitates a dedicated CPU core to execute the vhost process and manage VM I/O requests.

In summary, VFIO and NVMePass have no CPU overhead on the I/O path, while SPDK and virtio use CPU resources to process and submit I/O commands.

E. Fairness

This subsection evaluated fairness among multiple VMs when utilizing NVMePass. We assigned four CPU cores for each VM and ran 4k-randread-4-128 workloads.

Fig. 10a shows the overall IOPS of NVMePass with 2 SSDs in multiple VMs. For this experiment, we ran up to eight VMs, with VMs 0/2/4/6 utilizing one disk, while VMs 1/3/5/7 used the other disk. When running one or two VMs, the overall IOPS reaches the limit of one SSD (514.3K IOPS) and two SSDs (1028.5K IOPS), respectively. When we increase the number of VMs to 4 and 8, the overall IOPS of all VMs remains at the IOPS limit of 2 SSDs (approximately 1030K IOPS). Additionally, the performance of each VM was evenly distributed, which demonstrated that the resources are efficiently utilized and distributed fairly among all VMs.

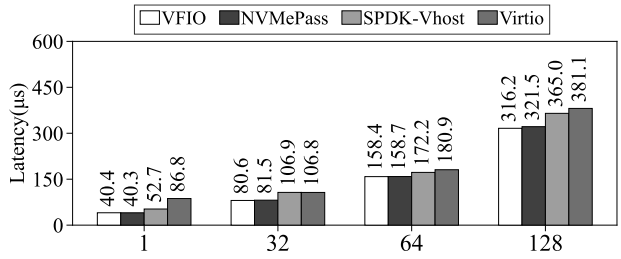
Moreover, we also examined the tail latency distribution of the eight VMs, as shown in Fig. 10b. We observe that the tail latency of each VM is closely distributed, which indicated that the storage resources are not tilted toward some VMs. This further illustrates that all VMs were receiving an equal share of storage resources.

In summary, NVMePass can maintain the overall throughput with VM increasing and guarantee the *fairness* of each VM.

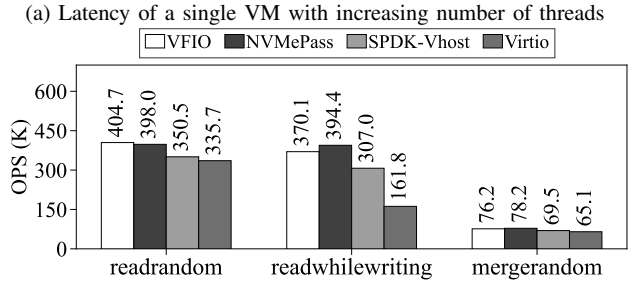
F. Real-world Application

For the real-world application, we evaluated the performance of RocksDB, a persistent key-value store for flash and RAM storage. We utilized db_bench to initiate I/O requests. For the single-VM, we assigned 4 CPU cores to each VM, while for the multi-VMs, we assigned one core for every 5 VMs. Specifically, we ran the "bulkload" case to insert 1 million data (with key size = 16 bytes, value size = 100 bytes, and total size = 1106.3 MB) into a database in sequential order.

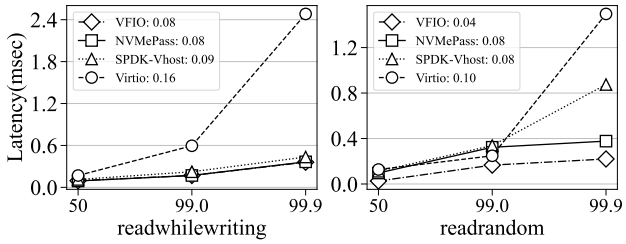
Single-VM. Firstly, Fig. 11a illustrates the latency (lower is better) of gradually increasing the threads from 1 to 128



(a) Latency of a single VM with increasing number of threads



(b) OPS of a single VM with different test cases using 128 threads



(c) Tail latency and average latency of single VM with 1 thread

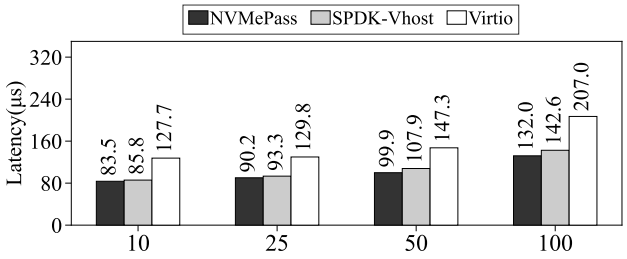
Fig. 11: Single VM running RocksDB workloads

in the readrandom workloads. NVMePass outperforms SPDK-Vhost and virtio and achieves performance close to VFIO. Specifically, under 128 concurrent threads, the average latency of NVMePass is only 1.7% higher than VFIO, and 11.9% and 15.6% lower than SPDK-Vhost and virtio.

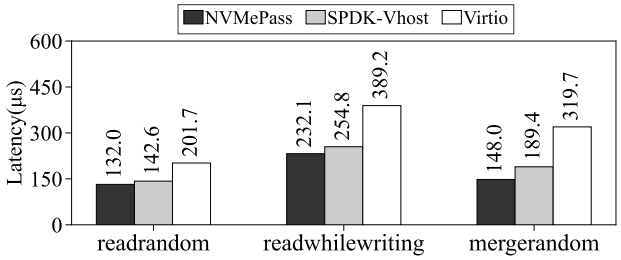
Secondly, we conducted readrandom, readwhilewriting, and mergerandom workloads on a single VM with 128 threads. The OPS (operations per second) are shown in Fig. 11b, where higher values indicate better performance. NVMePass achieves performance close to VFIO in all test cases. Concretely, in the readrandom case, the OPS of NVMePass is 98.3% of VFIO. Furthermore, NVMePass performs better than SPDK-Vhost and virtio in the readrandom and readwhilewriting cases. The OPS of NVMePass in readrandom and readwhilewriting is 13.6% and 28.5% higher than SPDK-Vhost and 18.6% and 143.7% higher than virtio, respectively.

Thirdly, Fig. 11c presents the tail latency results for the readwhilewriting and mergerandom workloads performed on a single VM with a concurrency of 1. These results indicate that NVMePass offers similar tail latency to VFIO and SPDK-Vhost and outperforms virtio. Virtio has the worst performance among the tested virtualization solutions due to its long I/O paths and context-switching overhead.

To summarize, NVMePass is a promising NVMe virtualization solution that offers a high throughput and low latency on a single VM in real-world workloads.



(a) Average latency of increasing VMs running readrandom with 1 thread



(b) Average latency of 100 VMs running various workloads with 1 thread

Fig. 12: Multiple VMs running RocksDB workloads

Multi-VMs. We conducted a readrandom workload with a concurrency of 1 on multiple VMs, gradually increasing the number of VMs up to 100. Fig. 12a demonstrates that NVMePass consistently maintains its performance advantage as the number of VMs increases. As the number of VMs rises from 10 to 100, NVMePass's total latency is consistently 2.6% to 7.4% lower than that of SPDK-Vhost. Notably, when the VM count reaches 25, NVMePass outperforms virtio by 30.5%, showcasing its superior scalability. These results demonstrate that NVMePass is a highly scalable solution that maintains its performance advantage in real-world workloads.

Then, we conducted different workloads with one thread on 100 VMs. The average latency of each VM is shown in Fig. 12b. Specifically, in the readrandom case, the latency of NVMePass is 7.4% lower than SPDK-Vhost. When compared with virtio, NVMePass also demonstrated better performance, with 34.6% and 40.3% lower latency in readrandom and readwhilewriting cases, respectively.

In summary, the results obtained from single and multiple VMs demonstrate that NVMePass outperforms both SPDK-Vhost and virtio in terms of OPS and latency when it comes to real-world workloads.

VI. Related Work

Software-based NVMe Virtualization. Virtio [47] is a series of well-maintained Linux drivers for general I/O device virtualization, but it suffers from significant performance degradation. SPDK vhost-NVMe [55] is an I/O service target that relies on user space NVMe drivers to eliminate unnecessary VM_Exit overhead and shrink the I/O execution stack in

the host OS. MDev-NVMe [43] is a mediated passthrough mechanism that utilizes a polling mode, and further research [42] has proposed an adaptive polling mode for MDev-NVMe to achieve better performance or save CPU resources for better scalability. FinNVMe [41] is a high-throughput storage virtualization management solution for an NVMe storage device that works in a workload-aware manner among multi-tenant VMs. Finally, Direct-Virtio [27] makes use of user-level storage direct access to avoid duplicated I/O stack overhead with polling methods. However, these polling-based solutions come at the cost of high CPU overhead.

Hardware-assisted NVMe Virtualization. SR-IOV [49] enables multiple VMs to share PFs and VFs of the device and achieve high performance. FVM [30] is a solution that builds on SR-IOV by offloading the virtualization layer to an FPGA card, which can directly manage the physical devices. BM-Store [16] relies on an FPGA-based BMS-Engine and an ARM-based BMS-Controller to achieve transparent and high-performance virtual local storage for bare-metal clouds. LeapIO [33] offloads the entire storage stack to an ARM SoC with support for both local and remote storage. vDPA (Virtio Data Path Acceleration) [21] is a framework that enhances the performance of virtio devices by offloading data path processing to dedicated hardware accelerators.

VII. Conclusion

In this paper, we propose NVMePass, a novel lightweight and software-hardware co-design NVMe passthrough virtualization architecture that offers high performance and no CPU overhead while maintaining high scalability. The key ideas behind NVMePass are a passthrough framework to allow VMs to directly access their I/O queues and a mechanism to ensure security. NVMePass is simple, efficient, and applicable for adoption, which has both the benefits of software and hardware-assisted virtualization mechanisms. We experimentally demonstrate that NVMePass provides comparable performance to VFIO and better performance than SPDK-Vhost.

VIII. Acknowledgment

This research was supported by Alibaba Group through the Alibaba Innovative Research Program and partially by the Research Grants Council of HKSAR (16213824). Jiexiong Xu and Wenzhi Chen are the corresponding authors.

References

[1] "Firecracker." [Online]. Available: https://firecracker-microvm.github.io/
[2] "Flexible i/o tester." 2023. [Online]. Available: https://github.com/axboe/fio
[3] "Non-uniform memory access (numa)," 2023. [Online]. Available: https://en.wikipedia.org/wiki/Non-uniform_memory_access
[4] K. Adams and O. Agesen, "A comparison of software and hardware techniques for x86 virtualization," *ACM Sigplan Notices*, vol. 41, no. 11, pp. 2–13, 2006.
[5] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications." in *NSDI*, vol. 20, 2020, pp. 419–434.
[6] Alibaba, "Alibaba cloud," 2023. [Online]. Available: https://www.alibabacloud.com/en
[7] Amazon, "Amazon web services," 2023. [Online]. Available: https://www.aboutamazon.com/what-we-do/amazon-web-services
[8] AMD, "Nvme target controller logicore ip product guide," 2021. [Online]. Available: https://docs.xilinx.com/r/en-US/pg329-nvme-target-controller/Command-Validation/Decode-Module
[9] AMD, "Amd i/o virtualization technology (iommu) specification," 2022. [Online]. Available: https://www.amd.com/en/support/tech-docs/amd-io-virtualization-technology-iommu-specification
[10] AMD, "Amd epyc™ 7773x," 2023. [Online]. Available: https://www.amd.com/en/products/cpu/amd-epyc-9654
[11] G. Ananthanarayanan, A. Ghodsi, S. Shenker, and I. Stoica, "Disk-locality in datacenter computing considered irrelevant," in *Proceedings of the 13th USENIX Conference on Hot Topics in Operating Systems*, ser. HotOS'13. USA: USENIX Association, 2011, p. 12.
[12] A. Awad, B. Kettering, and Y. Solihin, "Non-volatile memory host controller interface performance analysis in high-performance i/o systems," in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2015, pp. 145–154.
[13] M. Ben-Yehuda, M. D. Day, Z. Dubitzky, M. Factor, N. Har'El, A. Gordon, A. Liguori, O. Wasserman, and B.-A. Yassour, "The turtles project: Design and implementation of nested virtualization," in *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. Vancouver, BC: USENIX Association, Oct. 2010. [Online]. Available: https://www.usenix.org/conference/osdi10/turtles-project-design-and-implementation-nested-virtualization
[14] M. Ben-Yehuda, M. Factor, E. Rom, A. Traeger, E. Borovik, and B.-A. Yassour, "Adding advanced storage controller functionality via low-overhead virtualization." in *FAST*, vol. 12, 2012, pp. 15–15.
[15] Y. Chen, Y. Xie, Y. Wang, J. Xu, Z. Jin, A. Li, X. Fu, Q. Liu, and W. Chen, "Optimizing nvme storage for large-scale deployment: Key technologies and strategies in alibaba cloud," *IEEE Micro*, vol. 44, no. 5, pp. 47–56, 2024.
[16] Y. Chen, J. Xu, C. Wei, Y. Wang, X. Yuan, Y. Zhang, X. Yu, Y. Chen, Z. Wang, S. He *et al.*, "Bm-store: A transparent and high-performance local storage architecture for bare-metal clouds enabling large-scale deployment," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 1031–1044.
[17] N. Express, "Nvm express," 2023. [Online]. Available: https://nvmexpress.org/specifications
[18] A. Gavrilovska, S. Kumar, H. Raj, K. Schwan, V. Gupta, R. Nathuji, R. Niranjan, A. Ranadive, and P. Saraiya, "High-performance hypervisor architectures: Virtualization in hpc systems," in *Workshop on system-level virtualization for HPC (HPCVirt)*, 2007.
[19] Google, "Google cloud," 2023. [Online]. Available: https://cloud.google.com
[20] A. Gordon, N. Amit, N. Har'El, M. Ben-Yehuda, A. Landau, A. Schuster, and D. Tsafrir, "Eli: Bare-metal performance for i/o virtualization," *ACM SIGPLAN Notices*, vol. 47, no. 4, pp. 411–422, 2012.
[21] R. Hat, "Introduction to the vdpa kernel framework," 2020, accessed: 2024-07-30. [Online]. Available: https://www.redhat.com/en/blog/introduction-vdpa-kernel-framework
[22] Intel®, "Intel® virtualization technology for directed i/o," 2023. [Online]. Available: https://edc.intel.com/content/www/us/en/publications/specification-nuc12dcm-nuc12edb/intel-virtualization-technology-for-directed-i-o/
[23] M. Jung, "OpenExpress: Fully hardware automated open research framework for future fast NVMe devices," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 649–656. [Online]. Available: https://www.usenix.org/conference/atc20/presentation/jung
[24] S. Keeriyadath, "Nvme virtualization ideas for machines on cloud." https://www.snia.org/sites/default/files/SDC/2016/presentations/solid_state_storage/Sangeeth_NVMe_Virtualization_Ideas-rev.pdf.
[25] D. Kim, T. Yu, H. H. Liu, Y. Zhu, J. Padhye, S. Raindel, C. Guo, V. Sekar, and S. Seshan, "{FreeFlow}: Software-based virtual {RDMA} networking for containerized clouds," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019, pp. 113–126.
[26] S.-H. Kim, J. Shim, E. Lee, S. Jeong, I. Kang, and J.-S. Kim, "{NVMeVirt}: A versatile software-defined virtual {NVMe} device," in *21st USENIX Conference on File and Storage Technologies (FAST 23)*, 2023, pp. 379–394.
[27] S. Kim, H. Park, and J. Choi, "Direct-virtio: A new direct virtualized i/o framework for nvme ssds," *Electronics*, vol. 10, no. 17, p. 2058, 2021.

[28] S. Kumar, H. Raj, K. Schwan, and I. Ganev, "Re-architecting vmms for multicore systems: The sidecore approach," in *Workshop on Interaction between Opearting Systems & Computer Architecture (WIOSCA)*. Citeseer, 2007.

[29] J. Kwak, S. Lee, K. Park, J. Jeong, and Y. H. Song, "Cosmos+ openssd: Rapid prototype for flash storage systems," *ACM Trans. Storage*, vol. 16, no. 3, pp. 15:1–15:35, 2020. [Online]. Available: https://doi.org/10.1145/3385073

[30] D. Kwon, J. Boo, D. Kim, and J. Kim, "Fvm: Fpga-assisted virtual device emulation for fast, scalable, and flexible storage virtualization," in *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, 2020, pp. 955–971.

[31] A. Landau, M. Ben-Yehuda, and A. Gordon, "SplitX: Split Guest/Hypervisor execution on Multi-Core," in *3rd Workshop on I/O Virtualization (WIOV 11)*. Portland, OR: USENIX Association, Jun. 2011. [Online]. Available: https://www.usenix.org/conference/wiov11/splitx-split-guesthypervisor-execution-multi-core

[32] J. Lei, M. Munikar, K. Suo, H. Lu, and J. Rao, "Parallelizing packet processing in container overlay networks," *EuroSys 2021*, 2021.

[33] H. Li, M. Hao, S. Novakovic, V. Gogte, S. Govindan, D. R. Ports, I. Zhang, R. Bianchini, H. S. Gunawi, and A. Badam, "Leapio: Efficient and portable virtual nvme storage on arm socs," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 591–605.

[34] Z. Li, J. Cheng, Q. Chen, E. Guan, Z. Bian, Y. Tao, B. Zha, Q. Wang, W. Han, and M. Guo, "RunD: A lightweight secure container runtime for high-density deployment and high-concurrency startup in serverless computing," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 53–68. [Online]. Available: https://www.usenix.org/conference/atc22/presentation/li-zijun-rund

[35] J. Liu and B. Abali, "Virtualization polling engine (vpe) using dedicated cpu cores to accelerate i/o virtualization," in *Proceedings of the 23rd international conference on Supercomputing*, 2009, pp. 225–234.

[36] F. Manco, C. Lupu, F. Schmidt, J. Mendes, S. Kuenzer, S. Sati, K. Yasukata, C. Raiciu, and F. Huici, "My VM is Lighter (and Safer) than your Container," in *Proceedings of the 26th Symposium on Operating Systems Principles*. Shanghai China: ACM, Oct. 2017, pp. 218–233. [Online]. Available: https://dl.acm.org/doi/10.1145/3132747.3132763

[37] Microsoft, "Microsoft azure," 2023. [Online]. Available: https://azure.microsoft.com/en-us

[38] S. Motion, "Automotive ssd controller: Performance, reliability, and future trends," https://www.siliconmotion.com/download/DWbb/a/Automotive_SSD_CTRL_WP_EN.pdf, n.d., accessed: 2024-07-24.

[39] D. Narayanan, E. Thereska, A. Donnelly, S. Elnikety, and A. I. T. Rowstron, "Migrating server storage to ssds: analysis of tradeoffs," in *European Conference on Computer Systems*, 2009.

[40] K. W. Neo Jia, "Vfio mediated devices," 2016. [Online]. Available: https://docs.kernel.org/driver-api/vfio-mediated-device.html

[41] B. Peng, M. Yang, J. Yao, and H. Guan, "A throughput-oriented nvme storage virtualization with workload-aware management," *IEEE Transactions on Computers*, vol. 70, no. 12, pp. 2112–2124, 2021.

[42] B. Peng, J. Yao, Y. Dong, and H. Guan, "Mdev-nvme: Mediated pass-through nvme virtualization solution with adaptive polling," *IEEE Transactions on Computers*, vol. 71, no. 2, pp. 251–265, 2022.

[43] B. Peng, H. Zhang, J. Yao, Y. Dong, Y. Xu, and H. Guan, "MDev-NVMe: A NVMe storage virtualization solution with mediated Pass-Through," in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. Boston, MA: USENIX Association, Jul. 2018, pp. 665–676. [Online]. Available: https://www.usenix.org/conference/atc18/presentation/peng

[44] Y. Qiu, W. Yin, and L. Wang, "A high-performance and scalable nvme controller featuring hardware acceleration," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 5, pp. 1344–1357, 2022.

[45] RocksDB, "A persistent key-value store for fast storage environments," 2023. [Online]. Available: https://rocksdb.org

[46] M. Rosenblum and C. Waldspurger, "I/o virtualization: Decoupling a logical device from its physical implementation offers many compelling advantages." *Queue*, vol. 9, no. 11, p. 30–39, nov 2011. [Online]. Available: https://doi.org/10.1145/2063166.2071256

[47] R. Russell, "virtio: towards a de-facto standard for virtual i/o devices," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 95–103, 2008.

[48] Samsung Semiconductor, "Pm1733 nvme ssd brochure," 2024, accessed: 2024-07-21. [Online]. Available: https://download.semiconductor.samsung.com/resources/brochure/PM1733%20NVMe%20SSD.pdf

[49] P. SIG, "Single root i/o virtualization and sharing specification," 2010.

[50] A. Tavakkol, J. Gómez-Luna, M. Sadrosadati, S. Ghose, and O. Mutlu, "MQSim: A framework for enabling realistic studies of modern Multi-Queue SSD devices," in *16th USENIX Conference on File and Storage Technologies (FAST 18)*. Oakland, CA: USENIX Association, Feb. 2018, pp. 49–66. [Online]. Available: https://www.usenix.org/conference/fast18/presentation/tavakkol

[51] K. Tian, "Intel® scalable i/o virtualization."

[52] A. Williamson, "Irq bypass manager," 2015. [Online]. Available: https://lwn.net/Articles/653706/

[53] A. Williamson, "Vfio: A user's perspective." 2023. [Online]. Available: https://docs.huihoo.com/kvm/kvm-forum/2012/2012-forum-VFIO.pdf

[54] Q. Xu, H. Siyamwala, M. Ghosh, T. Suri, M. Awasthi, Z. Guz, A. Shayesteh, and V. Balakrishnan, "Performance analysis of nvme ssds and their implication on real world databases," in *Proceedings of the 8th ACM International Systems and Storage Conference*, ser. SYSTOR '15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: https://doi.org/10.1145/2757667.2757684

[55] Z. Yang, C. Liu, Y. Zhou, X. Liu, and G. Cao, "Spdk vhost-nvme: Accelerating i/os in virtual machines on nvme ssds via user space vhost target," in *2018 IEEE 8th International Symposium on Cloud and Service Computing (SC2)*. IEEE, 2018, pp. 67–76.

[56] YingRen, "Yingren dongting-n2," 2024, accessed: 2024-07-21. [Online]. Available: https://www.yingren.cn/ssd/dongting-n1/

[57] J. Zhang, M. Kwon, M. Swift, and M. Jung, "Scalable parallel flash firmware for many-core architectures," in *18th USENIX Conference on File and Storage Technologies (FAST 20)*. Santa Clara, CA: USENIX Association, Feb. 2020, pp. 121–136. [Online]. Available: https://www.usenix.org/conference/fast20/presentation/zhang-jie

[58] Z. Zhang, J. Chen, B. Ying, Y. Cao, L. Liu, J. Li, X. Zeng, J. Wang, W. Li, and H. Guan, "Hd-iov: Sw-hw co-designed i/o virtualization with scalability and flexibility for hyper-density cloud," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 834–850.